Test Quarto Website

Introduction

This is a tutorial for documenting research workflows using Quarto and RStudio. These protocols were developed in the context of systematic, biogeographic, and species delimitation studies using both morphometric and genetic data sets, but they could be employed for any research program, especially those that involve statistical analyses in R. We will perform a couple of analyses to illustrate some features of the documentation but this is not a tutorial on statistical analysis, rather it is an attempt to impart skills to document and share detailed workflows. The tutorial will assume some basic familiarity with statistical analysis and graphics in R.

To learn more about Quarto websites visit https://quarto.org/docs/websites.

Outline

- Create a GitHub repository.
- Create a new R project and Quarto website within R studio.
- Create an R environment.
- Edit the Quarto website settings to create pages documenting your workflows
- Include narrative, textual information and interactive code.
- Upload to GitHub repository.

Create a GitHub repository

Create a GitHub account

Go to https://github.com and create an account.

You should now have a dashboard with your user name and various options for customizing your GitHub dashboard, including options for creating a profile image, and links to various settings including options for creating new repositories.

Create a GitHub repository

Select `Repositories` from the top menu on your dashboard on github.com. This will take you to another page with a list of your repositories. You can create a single webpage for each GitHub repository.

Select the `New` button to create a new repository.

Enter an informative name in the `Repository name *` field along with a brief description in the `Description` field.

Choose `Public` from the dropdown menu for the `Choose visibility *` option under `Configuration`.

Click the toggle to `On` in the `Add README` option to generate an informative description of the repository. This should be a longer more detailed description than what is entered in the `Description` field.

The `No .gitignore` left as is will result in no `.gitignore` file being created. `.gitignore` is a text file in the directory that tells GitHub which file not to sync with the repository. This will be important later so you do want a `.gitignore` file in your home directory. You can create this file yourself later or allow GitHub to add it to the repository from a template. A long list of templates appear in the drop down menu. The best of these given we are working in RStudio and using primarily R for the analyses would be the R template. You can edit this file in the GitHub dashboard later to suit your project. For our purposes the `.gitignore` file should include these file names, with potentially more added later as needed. Note the `.env` file. This will be important later.

```
.env
.Rproj.user
.Rhistory
.RData
.Ruserdata

/.quarto/
.positai
```

For the `Add license` selection typically Creative Commons Zero v1.0 or some similar license allowing broad free public use is the best choice for most academic research. Often these open Creative Commons licenses are required for publicly funded research.

Click the `Create repository` button.

Edit the README.md file

You will now see the dashboard for your repository. All that will be in the repository for now is the `README.md` file. You may edit the contents of `README.md` in GitHub by clicking on it and clicking the `Edit` button. After you are finished editing the `README.md` file click `Commit changes...`. You can enter some brief description of the change if you like. Make sure the `Commit directly to main branch` option is selected and click `Commit change`. This will return you back to the main dashboard for the repository.

Edit the repository settings

Select `Settings` from the menu bar across the top of the page. The repository name should appear in the top field. Do not rename the repository. Also the `Default branch` field should read `main`.

On the left menu select `Pages` to set up your html webpage for this repository. This will take you to the settings for GitHub Pages. The `Source` option should read `Deploy from a branch` and for the `Branch` option select `main` from the dropdown menu and `/docs`. Click the `Save` button.

You now have a working GitHub directory and you are ready to deploy your workflow documentation and analyses. Continue to the next stage.

Create a new R project and Quarto website within R studio

You should have R, RStudio, and Quarto installed on your computer. You can conveniently install both the latest version of R and RStudio from the RStudio website.

Create a directory for your project on your computer.

Open RStudio and go to the `File` menu and select `New Project`. This will take you to the `New Project Wizard` in RStudio. Select `New Directory`.

After choosing `New Directory` you will see a list of project types. Select `Quarto Website`.

Click the `Browse` button and navigate to the directory you created specifically for this project. Name the project directory. Select the check boxes for the `Create a git repository`, `Use renv with this project`, and `Use visual markdown editor`.

Now click on the `Create Project` button.

Create an R environment

Working with an environment is a useful means to maintain consistency within an analysis such that the same package versions are used across sessions. If you selected the `Use renv with this project` when you initially set up the project in RStudio, congratulations, you do not have to do anything else. Your project will be running in its own dedicated environment. Remember that you will need to install packages into the research environments for each project you are working on as they will be stored in project-specific libraries. If you get a message during the creation of your project to install the `renv` package then add that package by following the prompts.

A note about entering code blocks in your Quarto webpage. The `/` key when selected at the beginning of a line will prompt a menu of choices including code blocks for different languages. Select the first one titled `R Code Chunk`. This creates a block on this line where you may not only enter computer code but also run that code block (or “chunk”) by hitting the triangle or “play” icon appearing in the top right of the block.

The language for that code block will appear in brackets on the first line (e.g., `{r}` denotes that block is running code in R). For each code block it is a good idea to include a brief description of the task performed for that particular block. Immediately below the `{r}` entry create a new line that begins with `#| label:`. This provides a unique identifier for each code block and enables the blocks to be cross referenced in the text and better organized. You do however need to be certain that the identifier for each block is unique. Duplicating the labels for the code blocks can create problems later. This is also helpful when there are problems as the offending block of code will be flagged by its label making the issue easier to trace. You may also add lines that begin simply with a hash (`#`) to identify additional information for the reader about that particular bit of code.

When in RStudio you may check the status of the environment by entering the following code in R.

```
renv::status()
```

```
No issues found -- the project is in a consistent state.
```

`No issues found — the project is in a consistent state` should be the correct response. If there is some inconsistency a snapshot of the environment may be taken with the following R code.

```
renv::snapshot()
```

```
- Lockfile written to "~/Library/Mobile
Documents/com~apple~CloudDocs/Marshall_University/Research/Pipelines and
protocols/Test/Test Quarto Website/renv.lock".
```

This action will update the project's lockfile which stores the current state of the dependencies in the project library.

Edit the Quarto website settings

You should now see your RStudio project. Open all the panes by clicking on `View`, then `Panes`, and `Show All Panes` if all the working panes are not visible. The source pane should be in the upper left quadrant of your screen. This is where you will enter text, images, and code to create your document. For editing your documents in the source pane you should choose the `Visual` editor option in the menu at the top of the source pane.

There are a number of other options here for editing text including bold (`B`), italics (`I`), and code (`</>`) selections for text. There is also a drop down menu beginning with `Normal` where you may select from either normal font for text or from a number of headings, which will create an outline of the document in the column on the right side of the source pane based on the headings. After this there are options for creating lists, inserting links and images, formatting text, and dropdowns for inserting code blocks, tables, citations, and other information. The menu bar above these options has choices for saving the file (a disc icon), rendering files to html documents, and running code blocks.

The source pane will initially will contain only two tabs corresponding to the two files created with the Quarto webpage. These will be titled `index.qmd` and `_quarto.yml`. Adding additional `qmd` files will ad additional

pages to the webpage and additional tabs in the source pane.

Select the `_quarto.yml` tab and in the project information, under `type: website`, type the following.

```
output-dir: docs
```

Be sure to click the `Save` icon on the file tab after making changes.

This will create a `docs` folder with the rendered files created by Quarto which will be uploaded to the GitHub repository.

Include narrative, textual information, and interactive code

Create a written narrative

This is the fun part. Be creative in generating the narrative of the workflow but most importantly be detailed in your instructions. The goal is that a reader should be able to follow your instructions and replicate your results exactly, or adapt your workflow to analyze their own data. There will be people with varying skill sets and experience reading your documentation, including sometimes students with very little background in coding or simply using a command line interface. Write your workflows with those people in mind.

Example: Sampling map

Here is an example of creating a sampling map for a phylogeography project.

First, install the packages needed for the analysis. This may be done as a command in R or in RStudio by selecting `Tools` form the menu bar, then `Install packages...`. Use the CRAN repository and search for the package you need to install and click on `Install`.

Packages

In your description of the analysis, list the needed packages. For this example we will use the following packages.

- ggplot2 (Wickham 2016)
- maps (Becker et al. 2025)
- sf (Pebesma and Bivand 2023)

Citations and bibliography

Quarto allows the user an easy way to insert citations, especially for R packages. The box at the top of your Quarto document allows for document specific formatting options. If you have multiple pages and therefore multiple Quarto files within a single webpage, most of these settings will be established globally in the `_quarto.yml` file (`.yml` files are called "*yaml*" file) so there typically will not be much at the top of each Quarto page. The `_quarto.yml` file should be visible as a tab in RStudio. However, for each page you will need

to assign a bibliography file (`.bib`). Remember the bibliographies for each of your pages should have their own unique `.bib` file.

Once you have assigned a page a unique `.bib` file you may start adding citations to the text. With the cursor in the place you wish to insert a citation, choose from the `Insert` menu on the menu at the top of the tab, and select `Citation...` . The citation feature of Quarto is not only useful for citing R packages but may be used to cite any published material from the scientific literature. For other sorts of citations references may be found by selecting `From DOI` or `PubMed` in the citation window. Adding citations will populate your bibliography file (`.bib`) and create a references section at the end of the rendered document. Sometimes references are not created in this file exactly as you might want them to appear. If this is the case you can edit the `.bib` file manually by opening in a text editor and making the needed changes.

Data files

Next have a data section that describes the data used in the analysis, highlights any particular facets of that dataset that could be confusing, defines any abbreviations relevant to reading the dataset, and alerts the reader to the file name and the location of that file. Also, it may be useful to create a blank code block and paste the raw data into that code block so that the reader may view those data easily without accessing the file itself.

List the files needed for the analyses. This sample analysis requires the following data file.

- `ZosteropsPOPS4_LatLong.txt` A text file of individual samples included in the SNP dataset matched with taxon, locale, and latitude and longitude.

Describe the data in detail so that it makes sense to the reader. For this example we have data from Asian white-eyes (*Zosterops spp.*). The population file lists the individual sample IDs, taxon designations, the name of the sampling locales (designated by island or province), and latitude and longitude will be used to generate a sampling map. To reduce the amount of clutter on the maps subspecies were clustered together in shared geographic groups. *Zosterops japonicus* subspecies were designated as either *Z. japonicus alani* for the most remote subspecies in the Ogasawara and Volcano Islands or grouped within a single *Z. japonicus* designation for all the remaining subspecies. Subspecies for *Z. montanus* were clustered into three groups including a Palawan group (montanus PALAWAN) consisting of *Z. montanus parkesi*, a northern Philippine group (montanus NORTH) for those subspecies on Mindoro and Luzon (*Z. montanus halconensis* amd *Z. montanus whiteheadi*), and a southern Philippine group (montanus SOUTH) including the remaining *Z. montanus* subspecies included in the study (*Z. montanus diuatae*, *Z. montanus montanus*, *Z. montanus pectoralis*, *Z. montanus vulcani*, and an unnamed but suggested subspecies on Panay). Subspecies within the *Z. everetti* and *Z. nigrorum* species were lumped together under a single species designation for each in the sampling maps but distinguished in later analyses. Samples from introduced populations in Hawaii and California are included in this list, but they are excluded from the final figure.

It is often helpful to show the data within the documentation. To do this select `Insert` and then `Code Block...` from the source pane menu. Do not enter anything for the `Language` field and this will create a blank block. You can paste your data into this block. For many datasets this is impractical because the files are too large. This is fine. You may just substitute displaying the data with a detailed description or only display the

partial data to give the reader an idea of what is in the data file. If you do the latter then make it clear to the reader that this is just a partial list. For our example, individual samples and their accompanying data from the file `ZosteropsPOPS4_LatLong.txt` are listed below. All of the samples listed in this file appear in the single nucleotide polymorphism (SNP) analyses.

```
SAMPLE  TAXON   LOCALE  LAT LONG
ZJlo012 japonicus   KIKAI   28.317  129.9401
ZJlo015 japonicus   AMAMI   28.3192 129.2765
ZJlo016 japonicus   MIYAKOJIMA  24.7916 125.3302
ZJlo017 japonicus   IRIOMOTE    24.3301 123.8188
ZJlo021 japonicus   TOKUNOSHIMA 27.7904 128.9668
ZJlo022 japonicus   TOKUNOSHIMA 27.7904 128.9668
ZJlo024 japonicus   YONAGUMI    24.4680 123.0045
ZJlo031 japonicus   KUMEJIMA    26.3504 126.7712
ZJlo045 japonicus   AMAMI   28.3192 129.2765
ZJlo046 japonicus   MIYAKOJIMA  24.8218 125.3137
ZJlo050 japonicus   OKINOERABU  27.5737 128.5737
ZJlo052 japonicus   OKINOERABU  27.5737 128.5737
ZJlo055 japonicus   KIKAI   28.317  129.9401
ZJja001 japonicus   GEOJE   34.8806 128.6211
ZJja002 japonicus   GEOJE   34.8806 128.6211
ZJja003 japonicus   JEJU    33.489  126.4983
ZJja004 japonicus   JEJU    33.489  126.4983
ZJja005 japonicus   JEJU    33.489  126.4983
ZJja009 japonicus   HEUKSANDO   34.6707 125.4196
ZJja010 japonicus   HEUKSANDO   34.6707 125.4196
ZJja012 japonicus   HONSHU  36.0835 140.0764
ZJja013 japonicus   HONSHU  36.0835 140.0764
ZJja014 japonicus   HONSHU  35.9086 139.6248
ZJja016 japonicus   SHIKOKU 34.3708 133.9198
ZJja030 japonicus   HEUKSANDO   34.6707 125.4196
ZJal003 japonicus alani HAHAJIMA    26.6735 142.1541
Zjal004 japonicus alani HAHAJIMA    26.6735 142.1541
ZJal005 japonicus alani HAHAJIMA    26.6735 142.1541
ZJal010 japonicus alani ANEJIMA 26.5506 142.1581
ZJal011 japonicus alani ANEJIMA 26.5506 142.1581
ZJal012 japonicus alani ANEJIMA 26.5506 142.1581
ZJal020 japonicus alani IWOTO 24.7876 141.3156
ZJst007 japonicus   KOZUSHIMA   34.2055 139.1345
ZJst009 japonicus   KOZUSHIMA   34.2055 139.1345
ZJst010 japonicus   KOZUSHIMA   34.2055 139.1345
ZJst012 japonicus   MIYAKEJIMA  34.0788 139.5178
ZJst014 japonicus   MIYAKEJIMA  34.0788 139.5178
ZJst015 japonicus   MIYAKEJIMA  34.0788 139.5178
ZJin001 japonicus   KAGOSHIMA   30.3715 130.6646
ZJin003 japonicus   KAGOSHIMA   30.3715 130.6646
ZJsi002 simplex TAIWAN  22.7613 121.1438
ZJsi017 simplex GUANGXI 22.8152 108.3275
ZJsi032 simplex GUANGXI 22.8152 108.3275
```

```
ZJsi023 simplex GUANGDONG   23.1317 113.2663
ZJsi024 simplex GUANGDONG   23.1317 113.2663
ZJsi027 simplex GUANGXI 22.8152 108.3275
ZJsi028 simplex GUANGXI 22.8152 108.3275
ZJsi029 simplex GUANGXI 22.8152 108.3275
ZJsi031 simplex GUANGXI 22.8152 108.3275
ZPxx001 simplex SINGAPORE   1.3189  103.8512
ZPxx002 simplex SINGAPORE   1.3521  103.8198
ZMxx002 meyeni  BATAN   20.4700 121.9910
ZMxx003 meyeni  LANYU   22.0811 121.5048
ZMxx004 meyeni  LANYU   22.0811 121.5048
ZMxx005 meyeni  LANYU   22.0811 121.5048
ZMxx006 meyeni  LANYU   22.0093 121.5721
ZMOxx001    montanus SOUTH  PANAY   11.4062 122.1235
ZMOxx002    montanus SOUTH  PANAY   11.4062 122.1235
ZMOxx003    montanus SOUTH  PANAY   11.3936 122.1661
ZMOxx005    montanus SOUTH  PANAY   11.4062 122.1235
ZMOha001    montanus NORTH  MINDORO 13.3082 121.0765
ZMOwh002    montanus NORTH  LUZON   15.5286 119.9608
ZMOwh003    montanus NORTH  LUZON   17.4670 121.0670
ZMOwh004    montanus NORTH  LUZON   15.4819 120.1194
ZMOwh005    montanus NORTH  LUZON   17.0373 121.1010
ZMOwh006    montanus NORTH  LUZON   16.5976 120.8986
ZMOvu002    montanus SOUTH  MINDANAO    7.8514  126.0458
ZMOvu003    montanus SOUTH  MINDANAO    7.8514  126.0458
ZMOvu004    montanus SOUTH  MINDANAO    7.0189  125.4984
ZMOvu005    montanus SOUTH  MINDANAO    7.0189  125.4984
ZMOmo001    montanus SOUTH  SULAWESI    -5.4081 119.8989
ZMOpa001    montanus PALAWAN    PALAWAN 8.8183  117.6750
ZMOpa002    montanus PALAWAN    PALAWAN 8.8183  117.6750
ZNni001 nigrorum    PANAY   11.4062 122.1235
ZERxx002    erythropleurus  RUSSIAb 46.0200 135.2500
ZERxx004    erythropleurus  RUSSIAb 44.9200 131.8900
ZERxx005    erythropleurus  RUSSIAb 44.3674 133.0265
Zsim23588   simplex VIETNAM 22.3860 102.2380
Zsim28142   simplex VIETNAM 21.9490 104.2550
Zsim30897   palpebrosus VIETNAM 15.1100 107.8240
Zpal23498   palpebrosus VIETNAM 22.3860 102.2380
Zpal23522   palpebrosus VIETNAM 22.3860 102.2380
Zmon20893   montanus SOUTH  NEGROS  10.6743 123.1886
Zmon20892   montanus SOUTH  NEGROS  10.6743 123.1886
Zmon20902   montanus SOUTH  NEGROS  10.6743 123.1886
Zmon20909   montanus SOUTH  NEGROS  10.6655 123.1787
Zsim31166   montanus SOUTH  JAVA    -7.6145 110.7122
Zsim31159   montanus SOUTH  JAVA    -7.6145 110.7122
Zmey17876   meyeni  BATAN   20.4700 121.9910
Zmey17877   meyeni  BATAN   20.4700 121.9910
Zmey17852   meyeni  BATAN   20.4570 121.9900
Zmey17922   meyeni  SABTANG 20.2850 121.8710
```

```
Zmey17920   meyeni  SABTANG 20.2850 121.8710
Zmey17925   meyeni  SABTANG 20.2850 121.8710
Zmey17923   meyeni  SABTANG 20.2850 121.8710
Zery28090   erythropleurus  VIETNAMnb   21.9490 104.2550
Zery28091   erythropleurus  VIETNAMnb   21.9490 104.2550
Zery28088   erythropleurus  VIETNAMnb   21.9490 104.2550
Zni10863    nigrorum    CAMIGUINNORTE   18.9290 121.8990
Zni14341    nigrorum    CAMIGUINSUR 9.1780  124.7197
Zni19650    nigrorum    LUZON   15.6436 121.5149
Zni17984    nigrorum    LUZON   14.0390 122.7860
ZjaDOT-10981    palpebrosus VIETNAM 22.7601 104.8805
ZjaDOT-5235 simplex TAIWAN  23.8280 120.7925
Z_LA_122866_2   simplex LOSANGELES  33.6966 -118.0183
Z_LA_122577_2   simplex LOSANGELES  33.6966 -118.0183
Z_LA_122188_2   simplex LOSANGELES  33.6966 -118.0183
Zsim13809   simplex GUIZHOU 25.4114 107.8872
Zsim13773   simplex GUIZHOU 25.4114 107.8872
Zsim6797    simplex HUNAN   28.4167 114.1167
Zsim10336   simplex GUANGXI 21.8400 107.8800
Zsim11362   simplex GUIZHOU 29.1668 107.5750
Zsim11102   simplex GUIZHOU 28.2261 107.1596
Zsim11220   simplex GUIZHOU 28.2261 107.1596
Zmon20899   montanus SOUTH  NEGROS  10.6743 123.1886
Zmon28375   montanus SOUTH  MINDANAO    9.0790  125.6650
Zev13949    everetti    CAMIGUINSUR 9.1925  124.7085
Zev31650    everetti    SAMAR   11.8280 125.2760
Zev28451    everetti    MINDANAO    9.0740  125.6420
Z_HI_BRY431 japonicus   HAWAII  19.5660 -155.4530
Z_HI_NAN290 japonicus   HAWAII  19.5660 -155.4530
Z_HI_WAI087 japonicus   HAWAII  19.5660 -155.4530
Z_HI_WAI078 japonicus   HAWAII  19.5660 -155.4530
Z_HI_SOL783 japonicus   HAWAII  19.5660 -155.4530
```

Analyses

Before starting your analyses, you will need to load the package libraries (make sure those packages are installed first). Remember to add a unique label (`#| label:`) at the top of each block of code (see **Create an R environment**). This may help with any troubleshooting downstream.

```
# Load in each library used for mapping
library(ggplot2)
library(maps)
library(sf)
```

```
Linking to GEOS 3.14.1, GDAL 3.5.3, PROJ 9.5.1; sf_use_s2() is TRUE
```

Once you have your packages installed and their libraries loaded, data are identified and described, and citations added where needed, now you are ready for some analyses. Remember these are examples just to

demonstrate what an analysis looks like in a Quarto webpage. This is not an R tutorial!

First, read your data into an R data frame and prepare the data frame to be used in a map. Data frames are used in R as the default means of storing tabular data (i.e., rows and columns). Remember you may run a particular block of code by clicking on the green triangle ("play" button) in the upper right hand corner of the code block.

```
# Read tab delimited ("\t") file with latitude and longitude
Zlocations <- read.csv("Data/Location data/ZosteropsPOPS4_LatLong.txt",
                       sep = "\t", header = TRUE)
# Remove the Los Angeles and Hawaii samples
Zlocations_NATIVE <- Zlocations[-c(105,106,107,120,121,122,123,124),]
# Split by TAXON
Zlocations_NATIVE_TAXON <- split(Zlocations_NATIVE,
                                 Zlocations_NATIVE$TAXON)
# init sampling.df which will be a df of samples grouped by unique lat/long
sampling.df<-data.frame(NULL)
# use a for loop to split the dataframe by species and summarize the number
# of individuals sampled at each locality for each species
for (i in names(Zlocations_NATIVE_TAXON)){
  samps <- Zlocations_NATIVE_TAXON[[i]] %>%
    dplyr::group_by(LAT, LONG) %>%
    dplyr::summarize(count=dplyr::n())
  df <- cbind(rep(i, times=nrow(samps)), samps)
  sampling.df <- as.data.frame(rbind(sampling.df, df))
}
```

```
`summarise()` has regrouped the output.
New names:
`summarise()` has regrouped the output.
New names:
`summarise()` has regrouped the output.
New names:
`summarise()` has regrouped the output.
New names:
`summarise()` has regrouped the output.
New names:
`summarise()` has regrouped the output.
New names:
`summarise()` has regrouped the output.
New names:
`summarise()` has regrouped the output.
New names:
`summarise()` has regrouped the output.
New names:
`summarise()` has regrouped the output.
New names:
`summarise()` has regrouped the output.
New names:
`summarise()` has regrouped the output.
New names:
ℹ Summaries were computed grouped by LAT and LONG.
```

```
ℹ Output is grouped by LAT.
ℹ Use `summarise(.groups = "drop_last")` to silence this message.
ℹ Use `summarise(.by = c(LAT, LONG))` for per-operation grouping
  (`?dplyr::dplyr_by`) instead.
```

```
#fix colnames
colnames(sampling.df)<-c("Species","Latitude","Longitude","count")
```

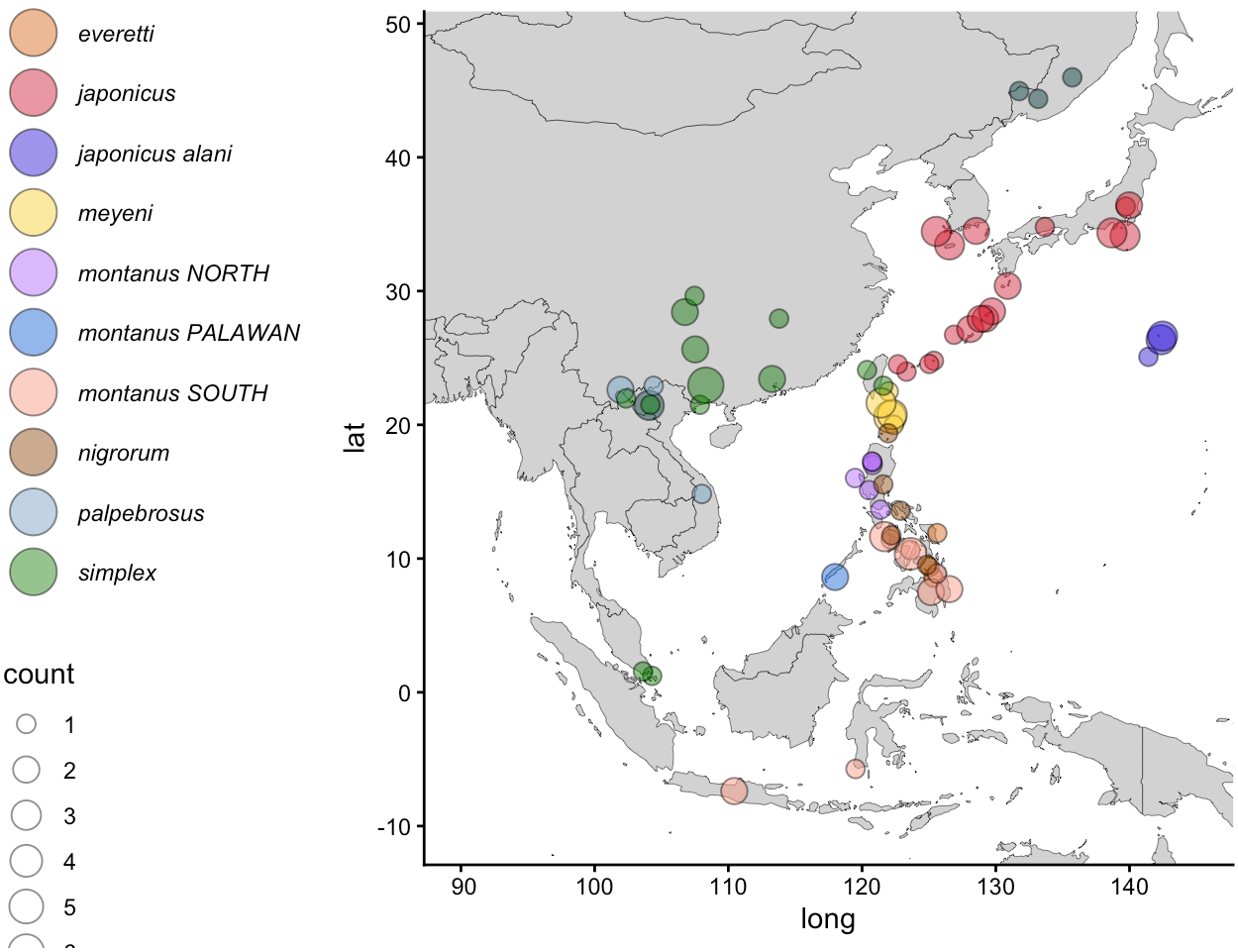
Next we will take the data frame we have just loaded and edited and use these location data to make a map of our samples. First get a base map for the world.

```
# make base world map
pac <- map_data("world")
```

Now that we have an object (`pac`) that contains our world map we can plot the samples onto the map, narrow its range to the area of interest, and set colors and other mapping options.

```
# plot all native samples
ggplot()+
  geom_polygon(data = pac, aes(x=long, y = lat, group = group),
               fill="lightgrey", col="black", cex=.1)+
  coord_sf(xlim = c(90, 145), ylim = c(-10, 48)) +
  geom_point(data = sampling.df, aes(x = Longitude, y = Latitude,
                                     fill=Species, size=count),
             position = position_jitter(h = 0.5, w = 0.5), pch=21,
             color="black", alpha =0.5, show.legend=TRUE) +
  theme_classic()+
  scale_fill_manual(values=c("#004949", "#DB6D00", "#D82632",
                             "#290AD8", "#FED439", "#B66DFF",
                             "#006DDB", "#FA9C88", "#924900",
                             "#7BA9CB", "#008600"))+
  scale_size_continuous(range = c(3,6))+
  guides(fill = guide_legend(override.aes = list(size = 8), order=1,
                             label.theme = element_text(face = "italic"),
                             position = "left"),
         linewidth = guide_legend(order = 2))+
  theme(legend.position = "left")
```

```
Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
ℹ Please use `linewidth` instead.
```



The map will now appear in your rendered html page or a rendered pdf.

Example: Principle component analysis of morphometric data and interactive output

One major benefit of rendering your workflow as an html page rather than simply a pdf is that the webpage format is interactive. Users can easily click the copy icon on a code block to copy code, scroll within a block if there are long lines of code, and move three dimensional plots other interactive graphics. Of course the drawback is that an html webpage needs to stable place to be hosted and is therefore not quite as portable as a pdf, but why not both! Hosting a workflow at GitHub is relatively easy and rendering your work as a webpage does not preclude you from also rendering as a pdf.

For this example we will take linear morphometrics from our Asian White-eyes and conduct a principle components analysis (PCA) and display the first three principle components as a 3D interactive figure.

General methods summary

Often it is good to start a particular analysis with a summary of the methodology. If you do this as you go along then once you are ready to write up your results for publication the methods section will be mostly completed already. Remember to be detailed. You have a little more freedom in supplemental material. You can create more detailed descriptions here compared to the methods section of a published paper. Here is an example outlining the methodology behind the Asian White-eye morphometrics example we will perform next.

Linear morphometric data including culmen length, nares-to-tip bill length, bill depth, bill width, tarsus length, wing cord, and tail length were collected from 388 round museum skins for all the Zosterops taxa in this study from the American Museum of Natural History (New York, NY, USA), Cincinnati Museum Center (Cincinnati, OH, USA), The Field Museum (Chicago, IL, USA), and the Smithsonian National Museum of Natural History (Washington DC, USA). All measurements were recorded by the same researcher (HLM) in mm using either digital calipers or a wing rule. Culmen length was measured as exposed culmen from the tip of the bill to the point where the bill meets the forehead plumage. Bill length from nares-to-tip was measured from the distal end of the right nare to the tip of the bill. Bill depth and width were measured at the nares. Tarsus (tarsometatarsus) length was measured from the base of the phalanges to the tibiotarsal joint. Wing cord was measured unflattened with a wing rule. Tail length was measured from the body to the tip of the longest rectrix by placing the same wing rule in the center of the tail. Skins where poor condition or positioning prevented accurate measurements for one or more traits were excluded from analysis making the final analyzed dataset 354 individual specimens. A PCA was conducted using the `prcomp` function and plotted for PC1-2 and for PC1-3 using *plotly* in R with specimens grouped solely according to species.

Packages

We will need the following additional packages. Remember when starting a new analysis provide the reader with a list of packages they will need and do not forget the citations.

- plotly (Sievert 2020)
- dplyr (Wickham et al. 2026)
- htmlwidgets (Vaidyanathan et al. 2023)

First we will load our packages. We will need to install the `plotly` and `htmlwidgets` packages. The `ggplot2` package is already installed and its library loaded as part of the previous analysis.

```
# Load the needed R packages (ggplot2 library has already been loaded)
library(plotly)
```

```
Attaching package: 'plotly'

The following object is masked from 'package:ggplot2':

    last_plot

The following object is masked from 'package:stats':

    filter

The following object is masked from 'package:graphics':

    layout
```

```
library(dplyr)
```

```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```
library(htmlwidgets)
```

Data files

The data arre in a comma separated text file (`csv`) named `Zosterops_linear_measurements2.csv` . Load the data into a data frame.

```
# Import data from CSV file
ZosteropsMorph <- read.csv("Data/Morphometrics/Zosterops_linear_measurements2.csv")
```

Now group specimens by species and select the variables we wish to analyze.

```
# Group data by 'species' and select relevant variables for PCA
ZosteropsMorph_species <- ZosteropsMorph %>%
  select(Species, BillCulmen, BillNareTip, BillDepth, BillWidth, Tarsus,
         Wing, Tail) %>%
  group_by(Species)
```

Analyses

Run the PCA using the `prcomp` function and save as a data frame called `pca_data` .

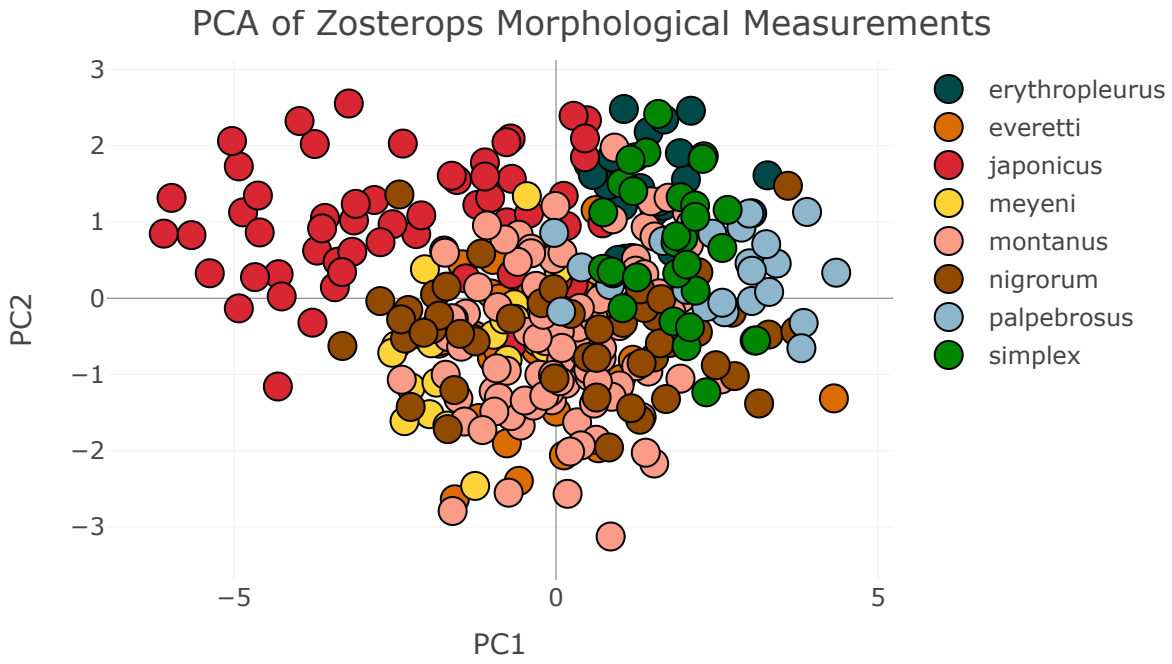
```
# Perform PCA on the selected variables
pca_result <- prcomp(ZosteropsMorph_species[, -1], center = TRUE,
                     scale. = TRUE)

# Convert PCA result to a data frame for plotting
pca_data <- as.data.frame(pca_result$x)

# Add 'species' variable to PCA data
pca_data$Species <- ZosteropsMorph_species$Species
```

Now plot the first two principle components.

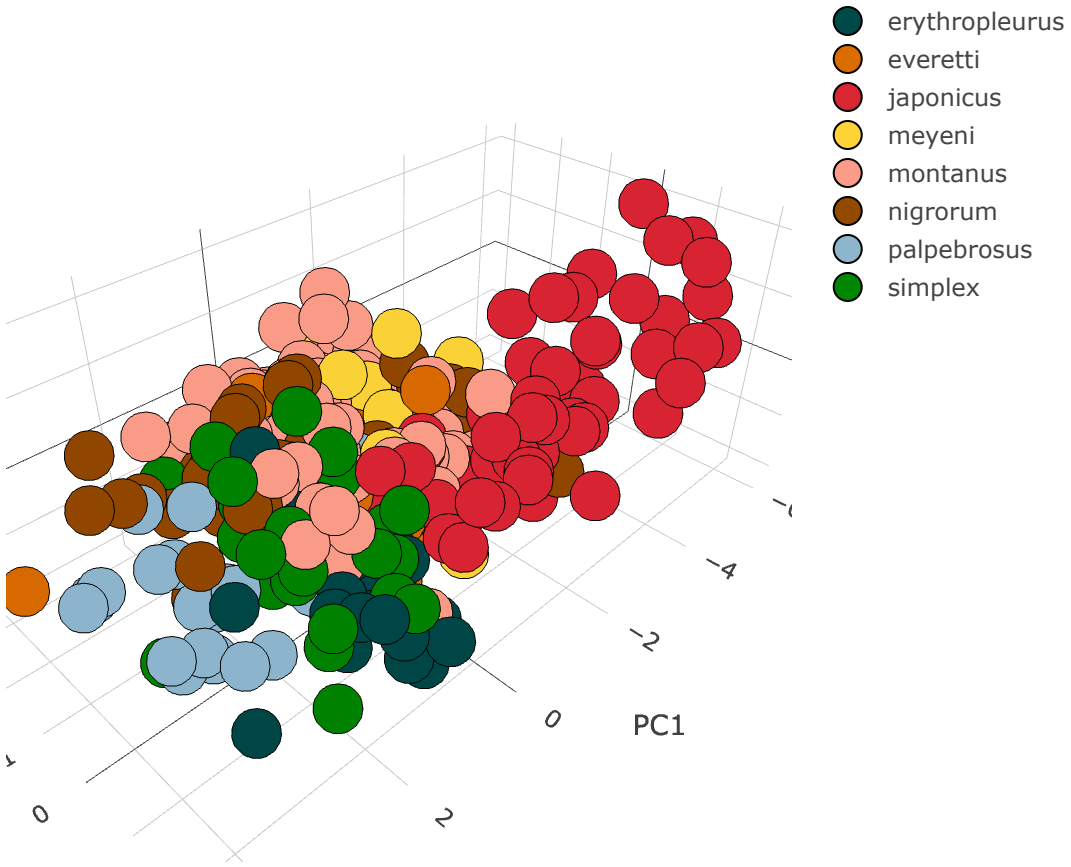
```
# Plot the first two principal components
ColorPal_CUSTOM <- c("#004949", "#DB6D00", "#D82632", "#FED439", "#FA9C88",
                     "#924900", "#8DB6CD", "#008600")
# Create the 2D scatter plot for PC1 and PC2 with outlines
PC2Dplot <- plot_ly(
  x = pca_data$PC1,
  y = pca_data$PC2,
  type = "scatter",  # Change to scatter for 2D plot
  mode = 'markers',
  marker = list(size = 14,
                line = list(color = 'black', width = 1)),  # Add black outline
  color = pca_data$Species,
  colors = ColorPal_CUSTOM
) %>%
layout(
  xaxis = list(title = 'PC1'),
  yaxis = list(title = 'PC2'),
  title = "PCA of Zosterops Morphological Measurements"
)
PC2Dplot
```



Now here is where things get interactive. Let's create a three dimensional graphic based on not the first two but the first three principle components. To do that we will execute the following code.

```
# Plot the first three principal components
PC3Dplot <- plot_ly(x = pca_data$PC1, y = pca_data$PC2, z = pca_data$PC3,
                    type = "scatter3d", mode = 'markers',
        marker = list(size = 14, line = list(color = 'black', width = 1)),
        color = pca_data$Species,
        colors = ColorPal_CUSTOM) %>%
  layout(scene = list(xaxis = list(title = 'PC1'),
```

```
                      yaxis = list(title = 'PC2'),
                      zaxis = list(title = 'PC3')))
PC3Dplot
```



Notice that the user may now interact with the graphic and move it around all three axes to view the data from different vantage points.

Adding a date/time stamp

You can add a date and time stamp to your `.qmd` files so that a reader may know the last update for that page. To set this up you need to include `knitr: true` in the front matter at the top of the `.qmd` file. In the editor in RStudio select the `Source` option as opposed to the `Visual` option and enter the following at the end of the document, or wherever you want the time/date stamp to appear.

```
*Last updated: May 20, 2026 14:22*
```

Render your results

Save results once a file has been updated by clicking on the save icon in the source pane menu, it appears as a computer disc icon.

Once your edits are completed then render the page by clicking on the `Render` selection in the source pane menu. Depending on the complexity of your page and the amount of code needed to run this can take a little while. Rendering will create a copy of your webpage in html format stored in the `docs` folder.

Upload to GitHub repository

Authenticate your GitHub remote repository

You need to specify where your files will be sent. Check to determine if there is a remote repository site set up by entering this command in the terminal.

```
git remote –v
```

If there is no output from this command then you need to set up the remote repository.

If you use the following code you will be asked for a username and password for your GitHub account. Remember throughout these instructions where you see `<USERNAME>` replace with your GitHub username and where you see `<REPOSITORY_NAME>` replace with the name you gave your GitHub repository.

```
git remote set-url origin github.com/<USERNAME>/<REPOSITORY_NAME>.git
```

However, despite getting prompts to enter a username and password, authenticating a remote repository on GitHub is no longer supported from command prompts in the terminal, so you will likely see the prompts but get an error message. Instead return to your GitHub account and create a token for your GitHub site.

Go to your dashboard for your GitHub account. Go to the `user navigation window` by clicking on your user icon in the top right corner. Select `Settings` from that menu. A list of menu options should be listed on the left side of your window. Scroll down and select the `< > Developer` settings option. Click on `Personal access tokens` on the left, from the drop down select `Tokens (classic)` and click the `Generate new token` button and from that drop down select `Generate new token (classic)`. You should now be on a new page that provides options for choosing the settings for the access token. Give your access token a name and from the token options select the `repo` and `workflow` check boxes. Now click on `Generate token`.

You have now created a security access token that may be used to upload changes to your GitHub site. The token is a long series of random characters like this (don't worry, this isn't an active token!)

```
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
```

Copy the token you have created on GitHub. You will not be able to go back and see it later. There is an icon beside it you can click on to copy.

There needs to be a secure place to store the token. We need to keep this token secret else errors will pop up when trying to push data to the GitHub site later so we will save the token in a `.env` file that will be ignored along with other files listed in a `.gitignore` file. Create a `txt` file in your text editor. The only contents of that file should be as follows, replacing <YOURTOKEN> with the token created on the GitHub site.

```
GITHUB_TOKEN=<YOURTOKEN>
```

Update Your remote URL setting in the terminal with this token. Replace <YOUR_USERNAME> with the username for your GitHub account and <YOUR_TOKEN> with the security token created on the GitHub site and matching the token in the `.env` file.

```
git remote set-url origin https://<YOUR_USERNAME>:
<YOUR_TOKEN>@github.com/<YOUR_USERNAME>/<REPOSITY_NNAME>.git
```

Install the `dotenv` package. You may do this in R by selecting `Tools`, followed by `Install packages...` and then search for `dotenv` and click on `Install`. Once you have the `dotenv` package installed load the package library and identify which file contains the GitHub token with the following commands in R.

```
library(dotenv)
load_dot_env()
github_token <- Sys.getenv("GITHUB_TOKEN")
```

Configure your GitHub repository

When you are ready to upload your changes to your GitHub website go to your terminal by selecting the `Terminal` tab in the lower left pane in RStudio.

Make certain that the name of the default branch is `main` in your GitHub `Pages` settings (see [**Create a GItHub repository**]).

```
git config --global init.defaultBranch main
```

Add your user email and GitHub username to your GitHub configuration.

```
git config --global user.email "youremail@gmail.com"
git config --global user.name "yourusername"
```

Verify that you are in the correct directory and check the status of your repository.

```
git status
```

Add files to the GitHub repository

To add new files or other changes the files need to be staged. Remember to add the period at the end of this command as that indicates changes are being added to the current directory.

```
git add .
```

Commit the staged changes and include a brief descriptive message. The first commit of data you may just call `Initial commit`. I sometimes use the date and time for descriptions of subsequent commits.

```
git commit –m "Initial commit"
```

After this commit you should see a list of files that were updated.

Now, you want to send or “push” these changes to the Github repository. First, check and make sure you have saved the right remote repository.

```
git remote –v
```

After confirming your remote repository conduct a pull from the repository to sync any changes that may exist on the repository that have not been saved locally. This is often unnecessary if you are the sole agent pushing changes to the repository but it is a good habit regardless.

```
git pull origin main
```

Next push your local changes to the GitHub repository. Make sure the location is `main`.

```
git push origin main
```

Note that the more explicit commands `git pull origin main` and `git push origin main` are probably more than you need. Unless you have your GitHub site set up to use multiple branches or remotes then simply `git pull` and `git push` are sufficient.

If your site is especially large and complex with multiple quarto files sometimes it may be better to push the changes for individual pages one at a time. Render the page from RStudio and instead of adding, commiting, and pushing everything all at once as described above do so individually for that single page. This requires staging the changes for the quarto file (`.qmd`), the html file (`.html`), and the directory created to store any additional files created during the render using the add command. For example, for the quarto file `ZosteropsMaps.qmd` first save your changes and render in RStudio using the `Render` button on the top menu. Once the Quarto file has been rendered and completed files saved in the `docs` folder add, commit, and push the changes to GitHub with the following commands (remember here we are using the `ZosteropsMaps.qmd` page as an example). Notice when updating an entire directory as opposed to an individual file you need to use quotes.

```
git add ZosteropsMaps.qmd
git add docs/ZosteropsMaps.html
git add "docs/ZosteropsMaps_files"
git commit –m "ZosteropsMaps update today's date"
git push
```

If the Quarto website is set up correctly you should always be rendering to a `docs` folder. For whatever reason sometimes this directory can get cluttered with duplicate files. You may start seeing files in that directory with numbered suffixes (e.g., `ZosteropsMaps_files2` and `ZosteropsMaps_files3`, etc.). If this happens you may manually delete all the files from the docs folder and conduct a fresh render on a clean directory. However, leave the file `.nojekyll` in this directory. This is a hidden, blank text file used in formatting your webpage. If you accidentally delete it then you can reconstitute it by creating a blank text file and saving it as `.nojekyll`. This is a hidden file in the directory (hence the `.` at the beginning of the file name) so normally

you will not see it unless you explicitly tell your computer to display hidden files (`Command + Shift + Period` in Mac OSX).

Render as a pdf

There are many benefits to distributing your workflows and data analyses through an html site including interactivity and ease in copying code. You may readily link html sites in the supplementary data accompanying a manuscript, or in the manuscript itself. However, html sites need to be hosted somewhere, like GitHub, and while these resources are fairly accessible, an author may prefer to share protocols, workflows, and data analyses in ways that do not require web hosting. Rendering a Quarto file as a pdf is an option in this case.

```
quarto render <YOUR _FILE_NAME>.qmd --to pdf
```

That command will work so long as your rendered Quarto document does not contain any complex html graphics and for this example we do have those graphics so this does not really work.

But there is an easy solution! Install the R package `webshot2` and load the package library. Your Quarto webpage is set up to render your document as an html file to be displayed on a browser. For each Quarto file (qmd) you create there will be a separate html file. Once the `webshot2` library is loaded you may use this package to convert that rendered html file into a pdf. It will take snapshots of html graphics to include in the pdf. Again, pdfs are simple to share over email or even as printed copies, they are not however interactive, so you will lose that feature of the html approach. But, it is good to have a pdf version of your workflow in addition to a webpage. If you wish your final product to be a pdf instead of a webpage you may set Quarto up in the beginning to render your documents as pdfs, just avoid using any complex, interactive graphics or html files and avoid long lines of code in your blocks as the reader will no longer be able to scroll left and right in a block in a pdf as they would in a webpage.

To convert your html file to a pdf using `webshot2` , execute the following commands in R. In the html version of your work this will even create a scrollable preview of the pdf embedded in the webpage.

```
library(webshot2)
webshot2::webshot(url = "https://hermmays.github.io/test/",
                  file = "docs/index.pdf")
```

```
https://hermmays.github.io/test/ screenshot completed
```

Last updated: May 20, 2026 14:22

References

Becker, Richard A., Allan R. Wilks, Ray Brownrigg, Thomas P. Minka, and Alex Deckmyn. 2025. “Maps: Draw Geographical Maps.” https://github.com/adeckmyn/maps.

Pebesma, Edzer, and Roger Bivand. 2023. “Spatial Data Science: With Applications in r.” https://doi.org/10.1201/9780429459016.

Sievert, Carson. 2020. “Interactive Web-Based Data Visualization with r, Plotly, and Shiny.” https://plotly-r.com.

Vaidyanathan, Ramnath, Yihui Xie, JJ Allaire, Joe Cheng, Carson Sievert, and Kenton Russell. 2023. “Htmlwidgets: HTML Widgets for r.” https://github.com/ramnathv/htmlwidgets.

Wickham, Hadley. 2016. “Ggplot2: Elegant Graphics for Data Analysis.” https://ggplot2.tidyverse.org.

Wickham, Hadley, Romain François, Lionel Henry, Kirill Müller, and Davis Vaughan. 2026. “Dplyr: A Grammar of Data Manipulation.” https://dplyr.tidyverse.org.